Template Risolutivi — Sistemi Operativi

Altri template, esercizi svolti e simulatori sono disponibili su so.rickywanga.com.

Esportazione degli appunti corrispondenti ai template del sito. Materiale di studio non ufficiale: i blocchi sono pseudocodice da adattare alle assunzioni e alla consegna, non programmi C compilabili né soluzioni certificate dal docente.

Appunti di Ricky Wanga · X @rickywanga42 · rickywanga.com

Schemi pronti da adattare durante l'esame.

C1 — Monitor

Semaforo Binario con Monitor

```
monitor monobinario {
  int value;
  int blockedCount = 0;
  condition block;

  monobinario(int v) { value = v; }

  monoP() {
    if (value == 0) {
      ++blockedCount;
      block.wait();
      --blockedCount;   // NOTA: OBBLIGATORIO dopo il risveglio (pattern
      ↪  waiting++/wait/waiting--)
    } else {
      value = 0;
    }
  }

  monoV() {
    if (value == 0 && blockedCount >= 1)
      block.signal();
    else
      value = 1;
  }
}
```

Lettori/Scrittori con Monitor

```
monitor ReadWrite {
  int readers = 0;
  int waitingWriters = 0;
  bool writing = false;
  condition ok2read, ok2write;

  procedure entry startRead() {
    if (writing || waitingWriters > 0) {
      ok2read.wait();
    }
    readers++;
    ok2read.signal();  // risveglia altri lettori in coda
  }

  procedure entry endRead() {
    readers--;
    if (readers == 0)
```

```
      ok2write.signal();
  }

  procedure entry startWrite() {
    waitingWriters++;
    if (readers > 0 || writing)
      ok2write.wait();
    waitingWriters--;
    writing = true;
  }

  procedure entry endWrite() {
    writing = false;
    if (waitingWriters > 0)
      ok2write.signal();
    else
      ok2read.signal();
  }
}
```

Produttore/Consumatore (Storage a Componenti)

```
monitor storage {
  int components[16];
  (condition, int) waiting[16];  // (coda, conteggio)

  procedure entry add(int[16] c) {
    for (int i = 0; i < 16; ++i) {
      components[i] += c[i];
      repeat(waiting[i].second)  // sveglia tutti in attesa
        waiting[i].first.signal();
    }
  }

  procedure entry get(int[16] requirements) {
    for (int i = 0; i < 16; ++i) {
      if (requirements[i] > components[i]) {
        ++waiting[i].second;
        while (requirements[i] > components[i])
          waiting[i].first.wait();
        --waiting[i].second;
      }
      components[i] -= requirements[i];
    }
  }
}
```

Rendez-vous “at least N”

```
monitor alrv {
  int waiting = 0;
  int target = 0;
  condition ok;

  procedure entry at_least(int n) {
    if (n <= 1) return;  // no attesa necessaria
    waiting++;
    if (waiting >= n) {
      target = n;
      repeat(n - 1) ok.signal();
      waiting -= n;
    } else {
```

```
      ok.wait();
    }
  }
}
```

C2 — Semafori

Semaforo Generale da Binari (Fair, FIFO)

```
class Semaphore {
  int value;
  int blocked = 0;
  binary_semaphore mutex(1);
  binary_semaphore sem(0);

  Semaphore(int init) { value = init; }

  void P() {
    mutex.P();
    if (value == 0) {
      blocked++;
      mutex.V();
      sem.P();       // si blocca qui
      blocked--;
    }
    value--;
    mutex.V();
  }

  void V() {
    mutex.P();
    value++;
    if (blocked > 0) {
      sem.V();       // sveglia un P in attesa
    } else {
      mutex.V();
    }
  }
}
```

Semaforo a Priorità LIFO

```
class OrderedLifoSemaphore {
  BinarySemaphore mutex(1);
  int initial, nP, nV = 0, 0, 0;
  OrderedStack<BinarySemaphore> s;

  OrderedLifoSemaphore(int init) { initial = init; }

  void PLP(int prio) {
    mutex.P();
    // NOTA: CORRETTO: initial - (nP+1) + nV < 0  (il +1 era un errore → faceva passare P con
    ↪  risorse già esaurite)
    if (initial - nP - 1 + nV < 0) {
      BinarySemaphore new_sem(0);
      s.push(prio, new_sem);
      mutex.V();
      new_sem.P();
    }
    nP++;
```

```
    mutex.V();
  }

  void PLV() {
    mutex.P();
    nV++;
    if (initial - nP + nV >= 0 && !s.empty())
      s.pop().V();   // sveglia priorità max (LIFO)
    else
      mutex.V();
  }
}
```

Passaggio del Testimone — Template Universale

Regola fondamentale: chi avvia la catena **NON rilascia la mutex**. La mutex viene passata di processo in processo insieme al testimone. L'ULTIMO risvegliato (counter == 0) fa finalmente `mutex.V()`.

```
binary_semaphore mutex(1);
semaphore s(0);
int counter = 0;

// ===== PROCESSO CHE SI BLOCCA =====
void attendi() {
    mutex.P();
    counter++;              // mi registro tra i bloccati
    mutex.V();              // RILASCIO PRIMA di bloccarmi
    s.P();                  // QUI MI BLOCCCO
    // ===== RISVEGLIATO =====
    counter--;
    if (counter > 0)
        s.V();              // PASSO IL TESTIMONE al prossimo
    else
        mutex.V();          // ULTIMO: RILASCIO LA MUTEX
}

// ===== PROCESSO CHE SBLOCCA TUTTI =====
void sblocca() {
    mutex.P();
    // ... operazioni sui dati condivisi ...
    s.V();                  // sveglia il PRIMO
    // NOTA: NON fare mutex.V()! La rilascerà l'ultimo risvegliato
}
```

wait4 — Blocchi di 4 (applicazione del template)

```
binary_semaphore mutex(1);
semaphore ok2go(0);
int n = 0;

void wait4() {
    mutex.P();
    n++;
    if (n >= 4) {
        n--;                // io NON sono tra i bloccati
        ok2go.V();          // sveglia il PRIMO. NON rilascio mutex!
    } else {
        mutex.V();          // rilascio prima di bloccarmi
        ok2go.P();          // mi blocco
        n--;                // RISVEGLIATO
        if (n > 0)
            ok2go.V();      // testimone al prossimo
```

```
        else
            mutex.V();       // ultimo: libero mutex
    }
}
```

sumstop/sumgo — Accumulo e Sblocco (applicazione del template)

```
int sum = 0;
semaphore s(0);
binary_semaphore mutex(1);

void sumstop(int v) {
    mutex.P();
    sum += v;
    mutex.V();               // rilascio PRIMA di bloccarmi
    s.P();                   // BLOCCATO
    // RISVEGLIATO:
    sum -= v;                // tolgo il MIO valore dalla somma
    if (sum > 0)
        s.V();               // testimone al prossimo
    else
        mutex.V();           // ultimo: libero mutex
}

int sumgo() {
    mutex.P();
    int tot = sum;           // catturo la somma
    s.V();                   // sveglia il PRIMO. NON rilascio mutex!
    return tot;
}
```

Barriera “All Out” (SAU)

```
binary_semaphore mutex(1);
semaphore all_out(0);
int in_section = 0, exiting = 0;

void SAU_enter() {
  mutex.P();
  in_section++;
  mutex.V();
}

void SAU_exit() {
  mutex.P();
  if (++exiting == in_section) {
    repeat(in_section - 1) all_out.V();  // sveglia tutti
    in_section = 0;
    exiting = 0;
    mutex.V();
  } else {
    mutex.V();
    all_out.P();    // aspetta gli altri
  }
}
```

C2 — Message Passing

Asincrono → Sincrono (con ACK)

```
void ssend(msg_t msg, pid_t dest) {
  asend((getpid(), msg), dest);
  (_, ack) = arecv(dest);   // aspetta ACK
}

msg_t srecv(pid_t sender) {
  (pid, msg) = arecv(sender);
  asend(ACK, pid);
  return msg;
}
```

Bloccante → Non-Bloccante (Dummy Messages)

```
bool skip = false;

T | None nbreceive(pid_t p) {
  if (!skip)
    asend((get_pid(), null), get_pid());  // dummy per sbloccare
  (pid, data) = areceive(p || get_pid());
  if (pid == get_pid() && data == null) {
    skip = false;
    return None;
  }
  skip = true;
  return data;
}
```

pssend/psreceive (message passing affidabile)

```
void pssend(T data, pid_t p) {
  asend((get_pid(), data), p);
  (pid_t, T) response;
  while ((response = areceive(p)) && response.second != ACK)
    asend(response, get_pid());  // reindirizza messaggi non-ACK
}

T | None psreceive(pid_t p) {
  res = nbreceive(p);
  if (res != None) {
    asend(ACK, res.first);
    return res.second;
  }
  return None;
}
```

mulsend — invio N copie

```
void mulsend(pid_t dest, T msg, int times) {
  for (int i = 0; i < times; ++i) {
    asend(dest, (msg, get_pid()));
    arecv(dest);  // aspetta ACK per ogni copia
  }
}

T multrecv(pid_t sender) {
  (msg, snd) = arecv(sender || ANY);
  asend(snd, ACK);
```

```
  return msg;
}
```

chained_send — invio a catena

```
void chained_send(T msg, list<pid_t> dests) {
  ssyncsend(msg, dests[0]);           // aspetta primo
  for (int i = 1; i < dests.size(); i++)
    ssyncsend(dests[i-1], dests[i]);  // ogni destinatario inoltra
}

T chained_recv(void) {
  msg = ssyncrecv(ANY);
  return msg;
}
```

G1 — Scheduling

Template SMP Biprocessore

Dati input tipici:

```
P1: cpu 4ms, I/O 4ms, cpu 2ms
P2: cpu 2ms, I/O 4ms, cpu 5ms
P3: cpu 5ms, I/O 3ms, cpu 3ms
P4: cpu 10ms, I/O 1ms

I/O: 1 unità, FIFO
Priorità: P1 > P2 > P3 > P4
```

Metodo: 1. Tracciare l'asse del tempo, due colonne CPU 2. All'inizio: processi ordinati per priorità sulle CPU libere 3. Quando un processo va in I/O: libera la CPU → schedulare il prossimo ready 4. Quando I/O completa: il processo torna ready → preemption se ha priorità > running 5. I/O è bloccante: il processo non può continuare finché I/O non completa 6. Context switch cost = 0 (salvo indicazione contraria)

Template Round-Robin Multilivello

```
Livello ALTO (FIFO): processi periodici H, K (ogni 6ms, 1ms CPU)
Livello BASSO (RR, q=3ms): P e Q (normali con I/O)

Regola: i processi ad alta priorità prevalgono SEMPRE
```

ED (Earliest Deadline) — Test di Schedulabilità

Per processi periodici con solo CPU:

```
Test di Liu & Layland: U = Σ(Ci/Ti) ≤ 1
  Ci = tempo CPU per istanza
  Ti = periodo

Se U > 1 → NON schedulabile
Se U ≤ 1 → DA VERIFICARE costruendo lo schedule
```

G2 — Domande Frequenti (Risposte Brevi)

Anomalia di Belady: Con algoritmi NON a stack (FIFO), aumentare i frame può aumentare i page fault. LRU è a stack → immune.

Working set: Insieme di pagine usate da un processo negli ultimi Δ riferimenti. Si calcola ad ogni page fault per decidere quali pagine tenere in memoria.

Trashing: Il sistema passa più tempo a fare swapping che a eseguire processi. Causato da troppi processi o poca memoria.

i-node e nome file: Il nome è nella directory, non nell'i-node. Così più nomi (hard link) possono puntare allo stesso i-node.

fsck: Controlla e ripara: (1) blocchi marcati liberi e usati, (2) link count errato, (3) blocchi inesistenti, (4) link simbolici rotti.

DMA vs terminali: I dischi trasferiscono grandi blocchi di dati → DMA evita di impegnare la CPU. I terminali trasferiscono pochi byte alla volta → overhead DMA non giustificato.

Buffer overflow: Scrivere oltre i limiti di un buffer in stack → sovrascrivere l'indirizzo di ritorno → eseguire shellcode iniettato.

Virus vs Worm: Virus = si attacca a un eseguibile, richiede azione utente. Worm = si replica autonomamente via rete.

`/etc/shadow`: Contiene hash delle password, accessibile solo a root. `/etc/passwd` è world-readable (contiene info utenti).

`noatime` su SSD: Gli SSD hanno numero limitato di scritture. Aggiornare `atime` a ogni lettura consuma cicli di scrittura inutilmente.

Quantum round-robin: Troppo piccolo → troppo context switch overhead. Troppo grande → diventa FIFO, perde reattività.

Banchiere: Algoritmo per evitare deadlock. Dato uno stato, verifica se esiste una sequenza safe di assegnazione risorse. Si esegue quando un processo richiede risorse. Se safe → assegna. Se unsafe → nega/blocca.